

Demand Forecasting

Google Cloud Deployment Guide

Step-by-step record of how the Demand Forecasting API, dashboard and daily forecasting job were deployed on Google Cloud (project **orlinappareldataset**, region **asia-south1 / Mumbai**), how they work together, how to operate and verify them, and how to recover from the problems met during deployment.

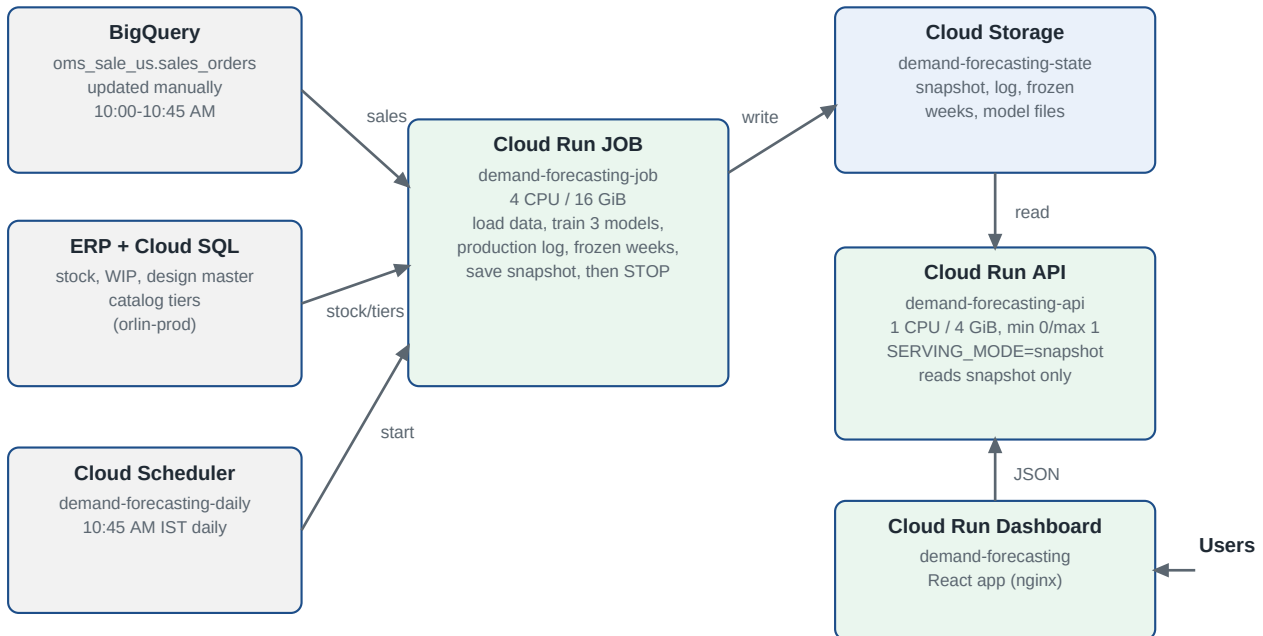
Item	Value
Deployed	26 September - 2 October 2026 (final setup completed 2 October 2026)
GCP project	orlinappareldataset (billing account 016C52-35C505-9F7877)
Region	asia-south1 (Mumbai)
Code repository	github.com/orlindatabase-lang/Demand-Forecasting (branch main)
Dashboard URL	https://demand-forecasting-174773483931.asia-south1.run.app
API URL	https://demand-forecasting-api-174773483931.asia-south1.run.app
Daily run	10:45 AM IST (Cloud Scheduler)
Expected cost	about Rs. 2,000 - 3,000 per month for this system

Contents

1. Architecture overview
2. Resource inventory
3. Prerequisites
4. Phase 0 - First deployment of API and dashboard
5. Phase 1 - Persistent storage (Cloud Storage)
6. Phase 2 - Daily job code
7. Phase 3 - Snapshot serving mode for a small API
8. Phase 4 - Cloud Run Job, scheduler, small API, build trigger
9. Verification checklist
10. Daily operations
11. Troubleshooting (problems met and fixes)
12. Rollback
13. Cost
14. Security notes and open items

1. Architecture overview

The system is split so that the expensive work runs only once a day. A **Cloud Run Job** does all the heavy work (load data, train the forecasting models, update the production log and frozen past weeks) and then stops. It saves the results as one **serving snapshot** file in Cloud Storage. A **small API** loads that file and answers the dashboard quickly. It sleeps when nobody uses it.



Daily flow

1. **10:00 - 10:45 AM** - the team updates the sales data in BigQuery (manual).
2. **10:45 AM** - Cloud Scheduler starts **demand-forecasting-job**.
3. The job restores saved files from the bucket, loads BigQuery + ERP data, retrains the 3 LightGBM models if the data is new, records the weekly production log, saves **serving_snapshot.pkl** and uploads everything to the bucket (about 3 - 15 minutes), then stops.
4. Within about 10 minutes the API sees the newer snapshot in the bucket and loads it. The dashboard then shows the new forecasts. No restart is needed.
5. The **Refresh** button on the dashboard starts the same job on demand.

What is stored where

Location	Contents	Written by
BigQuery: oms_sale_us.sales_orders	Raw sales orders	Team (manual update)
ERP report API	Stock, WIP, design master, bill of materials	ERP
Cloud SQL: orlin-prod / orlin-db	CatalogStyle tiers, tier history (read only by this app)	Admin platform
Cloud Storage: demand-forecasting-state	serving_snapshot.pkl, weekly_production_log.sqlite, forecast_freeze.sqlite, model forecast files, similar_design.json	Daily job only

2. Resource inventory

Resource	Name	Settings
Cloud Run service (API)	demand-forecasting-api	1 CPU, 4 GiB, instance-based billing, min 0 / max 1 instances, startup CPU boost on. Env: SERVING_MODE=snapshot, CACHE_BUCKET, CLOUDSQL_DB_USER, CLOUDSQL_DB_PASSWORD
Cloud Run service (dashboard)	demand-forecasting	1 CPU, 512 MiB, min 0 / max 3, port 8080 (nginx)
Cloud Run Job	demand-forecasting-job	4 CPU, 16 GiB, 1 task, timeout 3600 s, max 1 retry, command <code>python job.py</code> . Env: CACHE_BUCKET, CLOUDSQL_DB_USER, CLOUDSQL_DB_PASSWORD
Cloud Scheduler	demand-forecasting-daily	45 10 * * *, Asia/Kolkata, POST to the job's :run URL, OAuth as the compute service account
Cloud Storage bucket	demand-forecasting-state	asia-south1, Standard, uniform access, public access prevention enforced. Old file kept under old/
Artifact Registry	cloud-run-source-deploy	Image demand-forecasting-api (shared by API and job), image demand-forecasting (dashboard)
Build trigger (API + job)	demand-forecasting-api-deploy	Push to main, ignores dashboard/** and **/*.md. Steps: build, push, update service, update job
Build trigger (dashboard)	demand-forecasting-dashboard-deploy	Push to main touching dashboard/**. Steps: build, push, update service
Service account	174773483931-compute@dev-elooper.gserviceaccount.com	Used by API, job, scheduler and triggers. Has Cloud SQL Client, BigQuery, Storage Admin, Run Admin roles

3. Prerequisites

- Google Cloud SDK (gcloud) installed and logged in: `gcloud auth login` and `gcloud config set project orlinappareldataset`.
- Local Python venv at `D:\Demand_Forecasting\venv` with the API requirements installed (`venv\Scripts\pip install -r api\requirements.txt`).
- BigQuery access for local runs: `gcloud auth application-default login`.
- `api\env` with `CLOUDSQL_DB_USER` and `CLOUDSQL_DB_PASSWORD` (git-ignored; never commit it).
- Only **one person** changes the Cloud Run services at a time. Parallel edits overwrote settings several times during this deployment.

Copying commands: In the grey command boxes, long lines are wrapped to fit the page (continued lines are indented). When copying, join a wrapped line back into one line - especially in YAML, where a broken line makes the configuration invalid.

Important: The GitHub repository is public. Never put passwords or tokens in code or committed files. Set them as Cloud Run environment variables (or Secret Manager) instead.

4. Phase 0 - First deployment of API and dashboard

Goal: run the existing FastAPI backend and React dashboard on Cloud Run.

4.1 Code change: open the port first, load data in the background

The first deployments failed with *"container failed to start and listen on the port PORT=8080 within the allocated timeout"*. The API loaded all BigQuery/ERP data (several minutes) before opening its port. Fix (commit 54b9bef):

- **api/main.py** starts the web server immediately and starts the data load in a background thread. While loading, /health returns "status": "loading" and data endpoints return HTTP 503 "still loading".
- **dashboard/src/App.tsx** keeps retrying every 5 seconds while the API answers 503.

4.2 Deploy the API from the project folder

The Cloud SQL credentials were passed from api/.env through a temporary env file (deleted afterwards), so they never reached GitHub or the image.

```
cd D:\Demand_Forecasting
gcloud run deploy demand-forecasting-api --source . \
  --project orlinappareldataset --region asia-south1 \
  --cpu 2 --memory 8Gi --no-cpu-throttling --min-instances 1 --max-instances 1 \
  --timeout 900 --allow-unauthenticated \
  --env-vars-file <temporary-file-with-CLOUDSQL_DB_USER-and-CLOUDSQL_DB_PASSWORD>
```

4.3 Deploy the dashboard

```
gcloud run deploy demand-forecasting --source dashboard \
  --project orlinappareldataset --region asia-south1 \
  --port 8080 --cpu 1 --memory 512Mi --min-instances 0 --max-instances 3 --allow-unauthenticated
```

The dashboard image has the API URL built in (dashboard/Dockerfile, ARG VITE_API_URL). The API service name was kept as demand-forecasting-api so that URL stayed valid.

4.4 Build triggers (automatic deploy on git push)

Cloud Build -> Triggers -> Create trigger, source repository orlindatabase-lang/Demand-Forecasting, event *Push to a branch*, branch ^main\$, configuration *Inline*, service account 174773483931-compute@developer.gserviceaccount.com.

Trigger	File filter	Builds
demand-forecasting-api-deploy	Ignored files: dashboard/**, **/*.md	Root Dockerfile -> API service + daily job
demand-forecasting-dashboard-deploy	Included files: dashboard/**	dashboard/Dockerfile -> dashboard service

Dashboard trigger inline YAML:

```
steps:
- name: gcr.io/cloud-builders/docker
  args: ['build', '-t', 'asia-south1-docker.pkg.dev/orlinappareldataset/cloud-run-source-deploy/demand-forecasting:$COMMIT_SHA', '-f', 'dashboard/Dockerfile', 'dashboard']
- name: gcr.io/cloud-builders/docker
  args: ['push', 'asia-south1-docker.pkg.dev/orlinappareldataset/cloud-run-source-deploy/demand-forecasting:$COMMIT_SHA']
- name: gcr.io/google.com/cloudsdktool/cloud-sdk:slim
  entrypoint: gcloud
  args: ['run', 'services', 'update', 'demand-forecasting', '--image=asia-south1-docker.pkg.dev/orlinappareldataset/cloud-run-source-deploy/demand-forecasting:$COMMIT_SHA', '--region=asia-south1', '--quiet']
images: ['asia-south1-docker.pkg.dev/orlinappareldataset/cloud-run-source-deploy/demand-forecasting:$COMMIT_SHA']
options:
  logging: CLOUD_LOGGING_ONLY
  timeout: 1200s
```

The API trigger's final YAML is in section 8.5. The trigger description must be under 100 characters or the console refuses to save it.

5. Phase 1 - Persistent storage (Cloud Storage)

Problem: a Cloud Run container's disk is wiped on every restart, redeploy or scale-down. The production log, the frozen past-week forecasts and the trained models were lost each time, and cloud numbers drifted from local numbers.

5.1 Bucket

The existing bucket **demand-forecasting-state** (asia-south1) was reused. Its one old file was moved aside and public access prevention was enforced:

```
gcloud storage mv gs://demand-forecasting-state/naive_forecast_snapshot.json gs://demand-forecasting-state/old/naive_forecast_snapshot.json
gcloud storage buckets update gs://demand-forecasting-state --public-access-prevention
```

5.2 Copy the local history into the bucket (one time)

Stop the local API first, then:

```
cd D:\Demand_Forecasting\api\cache
gcloud storage cp weekly_production_log.sqlite forecast_freeze.sqlite similar_design.json gs://demand-forecasting-state/
gcloud storage cp lgbm_*2026-10-01.json gs://demand-forecasting-state/
gcloud storage ls gs://demand-forecasting-state/
```

5.3 Code (commit fc4f427)

- **api/cache_sync.py** (new): `download_recent()` copies the bucket's files into `api/cache` at start-up; `upload_changed()` uploads changed `.json`, `.sqlite` and `.pkl` files after each refresh. SQLite files are copied with SQLite's backup API first. Does nothing when `CACHE_BUCKET` is not set (local runs).
- **api/requirements.txt**: added `google-cloud-storage>=2.0`.
- **api/data.py**: download before the first load; upload after each rebuild and after training.

5.4 Enable it

Cloud Run -> service -> Edit & deploy new revision -> Variables & Secrets -> add `CACHE_BUCKET = demand-forecasting-state`.

Pitfall: The variable was first saved as `"CACHE_BUCKET "` (with a trailing space) and the feature silently stayed off. Type variable names by hand. Fix by command:

```
gcloud run services update demand-forecasting-api --region asia-south1 --remove-env-vars="CACHE_BUCKET "
--update-env-vars="CACHE_BUCKET=demand-forecasting-state"
```

Verified: the log showed `[cache_sync]` downloaded 5 files, the API served the trained model immediately, and all 108,985 completed-week forecast cells matched the local API exactly.

6. Phase 2 - Daily job code

Commit a37679a adds the job and the serving snapshot.

- **api/job.py** (new): `restore_cache_from_the_bucket -> data.rebuild()` (load data, publish plan) -> wait for model training (which also records the production log) -> `data.save_serving_snapshot()` -> upload changes -> exit (code 0 = success, 1 = data could not be loaded).
- **api/data.py**: `save_serving_snapshot()` writes everything the endpoints read (plan, forecasts, actuals, accuracy, festival effects and a slim copy of the gross sales rows) to **serving_snapshot.pkl** (about 276 MB); `load_serving_snapshot()` restores it.
- **api/cache_sync.py**: also syncs `.pkl` files.

Tested: every page built from the snapshot was identical to the page built from the full data; loading took about 1 second instead of 74 seconds and about 1.3 GB of memory instead of 8 - 16 GB.

Run the job locally

```
cd D:\Demand_Forecasting\api
..\env\Scripts\python.exe job.py
```

Expected last lines: [data] serving snapshot saved: serving_snapshot.pkl (276 MB ...) and
[job] done in N min. Locally (no CACHE_BUCKET) it only writes api\cache.

7. Phase 3 - Snapshot serving mode for a small API

Commit 23a0c33. With `SERVING_MODE=snapshot` the API:

- on start: restores the production log and frozen weeks from the bucket, downloads `serving_snapshot.pkl` and serves from it (no BigQuery load, no training);
- every 10 minutes (`SNAPSHOT_CHECK_SECS`): checks the bucket for a newer snapshot and loads it in the background;
- Refresh button (POST `/admin/refresh`): starts the Cloud Run Job (`DAILY_JOB_NAME`, default `demand-forecasting-job`) instead of rebuilding inside the API;
- never writes to the bucket - the job is the only writer.

Without `SERVING_MODE` the API behaves as before (full load + training), which is how it runs on the local PC.

Seed the bucket with a first snapshot:

```
gcloud storage cp D:\Demand_Forecasting\api\.cache\serving_snapshot.pkl gs://demand-forecasting-state/
```

8. Phase 4 - Cloud Run Job, scheduler, small API, build trigger

8.1 Create the job

```
gcloud run jobs create demand-forecasting-job \
  --project orlinappareldataset --region asia-south1 \
  --image asia-south1-docker.pkg.dev/orlinappareldataset/cloud-run-source-deploy/demand-forecasting-api:<COMMIT_SHA> \
  --command python --args job.py \
  --cpu 4 --memory 16Gi \
  --task-timeout 3600s --max-retries 1 --tasks 1 \
  --set-env-vars CACHE_BUCKET=demand-forecasting-state \
  --service-account 174773483931-compute@developer.gserviceaccount.com
```

Then **Cloud Run -> Jobs -> demand-forecasting-job -> Edit -> Variables & Secrets**: add `CLOUDSQL_DB_USER` and `CLOUDSQL_DB_PASSWORD` (same values as the API). Without them the models train with every tier set to "Unknown".

8.2 Run it once

```
gcloud run jobs execute demand-forecasting-job --project orlinappareldataset --region asia-south1 --wait
gcloud storage ls -l gs://demand-forecasting-state/serving_snapshot.pkl
```

First run: succeeded in 2.3 minutes (models reused); the snapshot timestamp updated.

8.3 Schedule it daily at 10:45 AM IST

```
gcloud scheduler jobs create http demand-forecasting-daily \
  --project orlinappareldataset --location asia-south1 \
  --schedule "45 10 * * *" --time-zone "Asia/Kolkata" \
  --http-method POST \
  --uri "https://run.googleapis.com/v2/projects/orlinappareldataset/locations/asia-south1/jobs/demand-forecasting-job:run" \
  --oauth-service-account-email 174773483931-compute@developer.gserviceaccount.com
```

Change the time later (example 11:15):

```
gcloud scheduler jobs update http demand-forecasting-daily --project orlinappareldataset --location asia-south1 --schedule "15 11 * * *"
```

Test immediately: `gcloud scheduler jobs run demand-forecasting-daily --project orlinappareldataset --location asia-south1`

8.4 Switch the API to the small size

Final settings: `SERVING_MODE=snapshot`, 1 CPU, 4 GiB, **instance-based** billing, min 0, max 1.

```
gcloud run services update demand-forecasting-api \
  --project orlinappareldataset --region asia-south1 \
  --update-env-vars SERVING_MODE=snapshot \
  --cpu 1 --memory 4Gi --no-cpu-throttling \
  --min-instances 0 --max-instances 1
```

Console alternative: service page -> **YAML** tab -> Edit, under `spec.template.metadata.annotations` set `autoscaling.knative.dev/minScale: '0'` and `run.googleapis.com/cpu-throttling: 'false'` (keep the quotes), then Save. In newer consoles Billing and Scaling are service-level settings on the Service details tab (pencil icons), not on the Edit & deploy page.

Pitfall: Request-based billing (`cpu-throttling: 'true'`) was tried first and the API never finished loading: Cloud Run only gives CPU while a request is being answered, so the background snapshot download stalled. Keep **instance-based** billing with **min 0** - the API still sleeps (Rs. 0) when nobody uses it.

8.5 Update the API build trigger so a push also updates the job

Cloud Build -> Triggers -> `demand-forecasting-api-deploy` -> Edit -> Configuration (Inline), replace the YAML with:

```
steps:
- name: gcr.io/cloud-builders/docker
  args: ['build', '-t', 'asia-south1-docker.pkg.dev/orlinappareldataset/cloud-run-source-deploy/demand-forecasting-api:$COMMIT_SHA', '-f', 'Dockerfile', '.']
- name: gcr.io/cloud-builders/docker
  args: ['push', 'asia-south1-docker.pkg.dev/orlinappareldataset/cloud-run-source-deploy/demand-forecasting-api:$COMMIT_SHA']
- name: gcr.io/google.com/cloudsdktool/cloud-sdk:slim
  entrypoint: gcloud
  args: ['run', 'services', 'update', 'demand-forecasting-api', '--image=asia-south1-docker.pkg.dev/orlinappareldataset/cloud-run-source-deploy/demand-forecasting-api:$COMMIT_SHA', '--region=asia-south1', '--quiet']
- name: gcr.io/google.com/cloudsdktool/cloud-sdk:slim
  entrypoint: gcloud
  args: ['run', 'jobs', 'update', 'demand-forecasting-job', '--image=asia-south1-docker.pkg.dev/orlinappareldataset/cloud-run-source-deploy/demand-forecasting-api:$COMMIT_SHA', '--region=asia-south1', '--quiet']
images: ['asia-south1-docker.pkg.dev/orlinappareldataset/cloud-run-source-deploy/demand-forecasting-api:$COMMIT_SHA']
options:
  logging: CLOUD_LOGGING_ONLY
  timeout: 1800s
```

Keep the 2-space indentation. Only the image changes on deploy; CPU, memory, variables and schedule stay.

9. Verification checklist

Check	How	Expected
API health	<code>https://demand-forecasting-api-174773483931.asia-south1.run.app/health</code>	<code>"status":"ok"</code> , <code>"dataReady":true</code> , <code>"forecastModel":"lightgbm"</code> (first call after idle may show <code>"loading"</code> for 15 - 30 s)
API settings	<code>gcloud run services describe demand-forecasting-api --region asia-south1</code>	1 CPU, 4Gi, <code>cpu-throttling false</code> , <code>minScale 0</code> , <code>maxScale 1</code> , <code>SERVING_MODE=snapshot</code>
Snapshot loaded	Cloud Run -> API -> Logs, search "serving snapshot"	<code>[data] serving snapshot loaded: data through ...</code>
Daily job ran	Cloud Run -> Jobs -> <code>demand-forecasting-job</code> -> Executions	A successful execution each day at about 10:45
Job log	Execution -> Logs	<code>[cache_sync] downloaded ...</code> , <code>[weekly_log] running week ... saved</code> , <code>[job] done in N min</code>

Check	How	Expected
Snapshot updated	gcloud storage ls -l gs://demand-forecasting-state/serving_snapshot.pkl	Time = today after 10:45
Dashboard	https://demand-forecasting-174773483931.asia-south1.run.app	Report, tiers, channel breakdown and Production Log load
Build	Cloud Build -> History after a push	4 green steps for the API trigger

10. Daily operations

Task	How
Normal day	Update BigQuery between 10:00 and 10:45. The 10:45 job and the API pick it up automatically.
Need fresh numbers now	Click Refresh on the dashboard (starts the job), or run <code>gcloud run jobs execute demand-forecasting-job --region asia-south1</code> . New numbers appear about 15 - 25 minutes later.
Late BigQuery update	Run the job again after the update (Refresh button), or move the schedule (section 8.3).
Deploy a code change	git push to main. The API trigger updates the API and the job; the dashboard trigger updates the dashboard when dashboard/ files change.
Run locally	<code>cd D:\Demand_Forecasting\api; ..\venv\Scripts\python.exe run.py</code> (full mode, port 8000). Local runs never touch the bucket.
Check cost	BigQuery table <code>billing_export.gcp_billing_export_v1_016C52_35C505_9F7877</code> , or Billing -> Reports (group by SKU: 'Jobs CPU/Memory' = job, 'Services CPU/Memory' = API and dashboard).

11. Troubleshooting (problems met and fixes)

Symptom	Cause	Fix
Deploy fails: container did not listen on PORT=8080 in time	Data loaded before the port opened	Background loading (commit 54b9bef)
Logs: Memory limit of 8192 MiB exceeded; API restarts in a loop	Full API training all 3 models needs more than 8 GiB	Training moved to the 16 GiB job; API serves the snapshot
Cloud forecasts differ from local; production log empty	Training never finished (memory / paused CPU); history wiped on restart	Phases 1 - 4
Tier filter and badges missing	Cloud SQL credentials not set (api/.env is not in the image)	Set CLOUDSQL_DB_USER / CLOUDSQL_DB_PASSWORD on the service and the job
Bucket sync silently off	Variable saved as "CACHE_BUCKET " with a space	Remove and re-add the variable (section 5.4)
API stuck on "loading" after switching to the small size	Request-based billing pauses CPU between requests	Instance-based billing with min 0 (section 8.4)
Push does not deploy the API	Trigger had only a build step / pointed at deleted services	Triggers recreated (section 4.4, 8.5)
Trigger cannot be saved in the console	Description longer than 100 characters	Shorten it
Settings change unexpectedly / trained state lost	Someone else redeployed or edited the service	One person deploys; check Logging audit logs for the editor
Extra job runs at odd times	Unknown caller of POST /admin/refresh from an internal Google address	Open item - see section 14
Job fails	BigQuery/ERP unreachable, permissions, code error	Execution -> Logs. The API keeps serving the previous snapshot meanwhile

12. Rollback

Back to the single full API (loads data and trains inside the API, about Rs. 33,000 per month):

```
gcloud run services update demand-forecasting-api --project orlinappareldataset --region asia-south1 --remove-env-vars SERVING_MODE --cpu 4 --memory 16Gi --no-cpu-throttling --min-instances 1
```

```
gcloud scheduler jobs pause demand-forecasting-daily --project orlinappareldataset --location asia-south1
```

Roll back a bad code version: Cloud Run -> service -> Revisions -> select an earlier revision -> Manage traffic -> 100% to it. For the job: `gcloud run jobs update demand-forecasting-job --region asia-south1 --image <earlier image>`.

13. Cost

Real costs from the billing export (INR, after credits). The project's normal baseline is about Rs. 300 - 400 per day, mostly the existing Cloud SQL instance (about Rs. 250 per day).

Period	Cloud Run	BigQuery	Note
26 Sep	Rs. 2,051	Rs. 117	First deployment
27 Sep	Rs. 8,279	Rs. 294	Up to 20 instances, CPU always allocated (about 47 CPUs non-stop)
28 Sep	Rs. 2,357	Rs. 139	Switched to request-based billing during the day
29 Sep - 1 Oct	Rs. 17 - 36 / day	Rs. 9 - 36 / day	Service mostly not running correctly
Total extra so far	about Rs. 12,700	about Rs. 450	About Rs. 13,000 up to 1 Oct

Expected monthly cost from now on

Part	Estimate per month
Daily job (4 CPU / 16 GiB, about 3 - 15 min per run)	Rs. 300 - 600
API (1 CPU / 4 GiB, about Rs. 11 - 12 per hour while awake, sleeps when idle)	Rs. 1,500 - 2,500
Dashboard, bucket, scheduler, builds	Rs. 0 - 100
Total	about Rs. 2,000 - 3,000 (previously about Rs. 33,000 for the always-on 16 GiB API)

Cloud Run Jobs have no request-based / instance-based option: they are billed only while an execution runs. In billing reports the job appears as 'Jobs CPU / Jobs Memory in asia-south1'. Recommended: create a budget alert under Billing -> Budgets & alerts (for example Rs. 5,000 per month).

14. Security notes and open items

Security

- The GitHub repository is **public**. `api/live_source.py` contains the ERP report token and server address as defaults. Rotate the ERP token, move it to a Cloud Run environment variable, and consider making the repository private.
- Cloud SQL credentials are set only as Cloud Run environment variables (service and job) and in the local, git-ignored `api/.env`. Consider moving them to Secret Manager.
- Both Cloud Run services allow unauthenticated access. Anyone with the API URL can call it, including `POST /admin/refresh`, which starts a job run.

Open items

- Unknown /admin/refresh caller**: an internal Google address (169.254.169.126) calls the endpoint; each call now starts a job run. Check the admin platform, old Cloud Scheduler entries and uptime checks.
- Parallel job runs**: Cloud Run Jobs do not block a second execution. A Refresh click during the 10:45 run starts a second run (harmless, about Rs. 10 - 20). A guard in the Refresh endpoint can be added.
- Old triggers**: `rmgpgab-demand-forecast-dashboard-...` and other `rmgpgab-demand-forecast*` triggers created outside this guide may still exist - review Cloud Build -> Triggers and delete unused ones.
- First scheduled run**: confirm the 3 October 10:45 execution succeeded.
- Optional**: copy the production log into Cloud SQL for querying from the admin platform; reduce memory use further; switch `xgboost` to `xgboost-cpu` to shrink the image by about 300 MB.

Commit history of this deployment

Commit	Change
54b9bef	Load data in background at startup; freeze completed-week forecasts
fc4f427	Keep api/.cache in Cloud Storage (cache_sync.py)
a37679a	Daily job (job.py) and serving snapshot
23a0c33	Snapshot serving mode for the small API